GL_from_WEBGL

Detailed code and rendering-pipeline explanation

Scope. This document explains the code currently present on the **main** branch of `D:\programming\git_rsaroha\GHCP\GL_from_WEBGL`. It covers the Win32/OpenGL host, model import, framebuffer design, GLSL shaders, Jump Flood Algorithm (JFA), effects, user input, and build/runtime limitations.

Executive summary

This project is a native Windows OpenGL 3.3 port of a WebGL distance-field effects demo. It renders a 3D mesh twice: first as a normally lit scene, and again as a silhouette used to construct a signed screen-space distance field. The distance field is then visualized as one of eight effects such as an outline, glow, pulse, halftone, or rings. Dear ImGui provides controls for model loading and effect parameters.

Input	Processing	Output
Procedural torus knot by default; optional OBJ, glTF, or GLB triangle mesh.	Scene render -> low-resolution mask -> mask dilation -> signed seed field -> ping-pong JFA -> effect composite.	Interactive Win32 window with lit model, selected effect, ImGui panel, and optional 4x MSAA.

Key source locations

Area	File / lines	Purpose
Application host	`main.cpp` 1-1957	Window creation, OpenGL loading, resources, rendering, UI, input, loop, cleanup.
Shader programs	`shaders/*.vert` and `*.frag`	Mesh transform, lighting, masks, field propagation, and effect compositing.
Build definition	`CMakeLists.txt`	Win32 executable, C++17, ImGui/tinygltf/tinyobjloader, shader copy step.
User-facing summary	`README.md`	Build commands, supported formats, controls, and known limitations.

1. Program architecture

The program is organized as one C++ translation unit plus standalone GLSL files. The C++ side owns the Win32 window, OpenGL context, GPU objects, model data, render targets, UI state, and frame loop. The shaders contain the per-vertex and per-pixel work. OpenGL 3.3 entry points beyond the legacy API are loaded with `wglGetProcAddress` in `loadGL`.

The main runtime state is global because the application has one window and one active mesh. Important state includes window dimensions, effect mode, thickness, camera yaw and pitch, image pan/zoom, inside/outside selection, auto-rotation, and program/target handles.

Frame-level control flow

Step	Function	What happens
1	`PeekMessageW` / `wndProc`	Windows messages update camera, zoom, thickness, effect mode, color, rotation, close state, and resize targets.
2	`render(time)`	Runs the complete scene, mask, distance-field, effect, and resolve pipeline.
3	`renderUi`	Builds the ImGui controls and draws them into the presentation framebuffer.
4	`presentPass`	Blits the completed presentation texture to the default window framebuffer.
5	`SwapBuffers`	Displays the frame using the Win32 double-buffered device context.

Coordinate systems

The mesh is transformed by a perspective projection and a look-at view matrix. `M4` stores column-major 4x4 matrices, matching GLSL/OpenGL conventions. `model.vert` applies the combined `viewProj` matrix, then applies screen-space `imageZoom` and pixel-based `imagePan`. Orbiting changes the 3D camera, while pan and zoom adjust the projected image.

Resource ownership

`Mesh` contains a VAO, vertex buffer, index buffer, and index count. `Target` owns a single-sample texture-backed framebuffer, optionally with a depth texture. Two targets are used for field ping-ponging and two for the low-resolution mask. `MultisampleTarget` owns the optional multisample color/depth textures used for scene/effect output before resolve.

2. Startup and OpenGL initialization

Startup begins in `wWinMain` around line 1852. The process is marked DPI-aware, then `createWindow` creates a Win32 window, selects a pixel format, creates a temporary bootstrap context to access WGL extension entry points, and finally creates an OpenGL 3.3 core-profile context with `wglCreateContextAttribsARB`.

After making the final context current, the application loads required OpenGL functions. It queries `GL_MAX_SAMPLES` and chooses up to four samples for multisample output. It creates the presentation target, loads shader files relative to the executable, compiles and links seven programs, creates a full-screen-triangle VAO, and creates the procedural torus-knot fallback.

Program	Vertex shader	Fragment shader	Use
litProgram	model.vert	lit.frag	Opaque scene shading with depth.
maskProgram	model.vert	mask.frag	Low-resolution silhouette.
seedProgram	model.vert	seed.frag	Signed surface seed into field[0].
seedQuadProgram	quad.vert	seed_quad.frag	Initialize field with far sentinel.
expandProgram	quad.vert	expand.frag	Dilate mask by one texel per pass.
jfaProgram	quad.vert	jfa.frag	Nearest-seed propagation.
effectProgram	quad.vert	effect.frag	Distance-field visualization and composite.

Error handling

Shader file failures, shader compilation/link failures, incomplete framebuffers, and startup failures use a Windows message box followed by process exit where the program cannot continue. During rendering, `glDebug` drains OpenGL errors and writes diagnostic messages with the pass name and beginning/end phase to debugger output.

Build configuration

`CMakeLists.txt` requires Windows, builds a C++17 Win32 executable, links `opengl32`, `user32`, `gdi32`, and `comdlg32`, and includes Dear ImGui plus repository third-party headers. A post-build command copies `shaders` beside the executable, which is required by runtime shader loading.

3. Model creation and import

Procedural fallback. `makeMesh` around line 588 generates a torus-knot-like tube using 256 rings and 32 sides. It samples a centerline, estimates a tangent and local frame, sweeps a circular cross-section, computes normals, converts coordinates to the renderer's orientation, and emits indexed triangles.

Common GPU layout. Imported and procedural geometry use six floats per vertex: position XYZ followed by normal XYZ. `uploadMesh` binds attribute 0 to position and attribute 1 to normal with a stride of six floats. Every draw uses indexed `GL_TRIANGLES`.

OBJ path. `loadObj` around line 867 configures tinyobjloader to triangulate faces. It validates each face and vertex index, reads positions and optional normal indices, and appends triangles. If a face has no usable normals, `appendTriangle` generates a face normal from two edges.

glTF/GLB path. `loadGltf` around line 1077 selects ASCII glTF or binary GLB loading. It traverses the selected scene, recursively combines node transforms, validates mesh primitives, reads POSITION and optional NORMAL accessors, supports indexed and non-indexed triangles, applies inverse-transpose normal transformation, and appends transformed geometry. Cycles, invalid nodes, unsupported primitive modes, sparse accessors, invalid strides, and buffer overruns are rejected explicitly.

Normalization. `normalizeModel` around line 807 calculates an axis-aligned bounding box, centers geometry at its midpoint, finds the maximum radius, and scales the result so the radius is approximately 0.9. It rejects missing triangles, non-finite coordinates, and zero-size bounds.

Supported	Not supported / intentionally omitted
OBJ, glTF, GLB; triangle primitives; positions; optional normals; glTF node transforms; indexed or non-indexed triangle geometry.	Materials and textures; animation; sparse accessors; non-triangle glTF primitives; HDR environment lighting; image-based lighting.

Load transaction. The ImGui load action first loads into a temporary `Mesh candidate`. The active mesh is replaced only after parsing, validation, normalization, and GPU upload succeed. On failure, the current mesh remains active.

4. Render-target and pass design

The renderer uses texture-backed framebuffers so intermediate images are available as sampler inputs to later full-screen passes. Targets use nearest filtering and clamp-to-edge wrapping, appropriate for discrete mask and distance-field texel operations.

Target	Format	Resolution	Role
presentationTarget	RGBA8 + depth texture	Window size	Single-sample scene/effect/UI composition before final presentation.
multisampleOutput	RGBA8 + depth texture	Window size, up to 4 samples	Optional MSAA destination for opaque scene and effect composite.
field[0], field[1]	RGBA32F; field[0] has depth	Window size	Ping-pong signed seed coordinates and distances.
mask[0], mask[1]	R8	Window size / thickness	Low-resolution silhouette and expanded mask.

Why the mask is lower resolution. Mask dimensions are `floor(width / thickness)` by `floor(height / thickness)`, clamped to at least 3 pixels. Larger thickness values reduce the mask and field working resolution. This lowers cost and gives the effect a stylized pixel scale while the full-resolution scene remains separate.

Pass order for every frame

#	Pass	Destination / source	Result
1	Scene	MSAA output or presentationTarget	Gray-blue lit mesh on gray background with depth testing.
2	Mask	mask[0]	Binary mesh silhouette at reduced resolution.
3	Mask expansion x4	mask[0] <-> mask[1]	Four one-texel dilations create a padded silhouette.
4	Seed initialization	field[0]	Far sentinel everywhere, then mesh fragments write signed surface seeds.
5	Jump flood	field[0] <-> field[1]	Nearest signed seed propagated over the working area.
6	Effect	Same output target as scene	Full-screen alpha composite selected from signed distance.
7	Resolve	MSAA -> presentationTarget	Single-sample color resolve when FSAA is active.
8	UI	presentationTarget	ImGui controls and runtime statistics.
9	Presentation	presentationTarget -> default framebuffer	Blit to the window and swap buffers.

5. Signed distance field and Jump Flood Algorithm

The field texture stores a compact record in RGB: R,G are the screen-space coordinates of the nearest seed, and B is the signed distance from the current pixel to that seed. Alpha is set to 1 for valid results. A blue value of zero is treated as uninitialized by the JFA shader.

Seed initialization. `renderSeedPass` first draws `seed_quad.frag` over field[0]. That shader writes the current fragment coordinate and a large positive distance (10000). It then draws the mesh with `seed.frag`, which writes the fragment coordinate and a large negative distance. The negative sign marks geometry-covered pixels as inside/negative seeds while the positive sentinel represents outside/uninitialized space. Depth testing makes the nearest visible surface win.

Jump flooding. `jumpFloodPass` starts with a step equal to `min(max(width,height), thickness)`. Each iteration samples up to eight neighbors at offsets of `(-1..1) * step`. It compares candidates and writes the best record to the opposite field target. The step is halved with ceiling until it reaches one. This spreads nearest-seed information rapidly instead of requiring a full search per pixel.

Signed comparison. In `jfa.frag`, `sg = sign(result.z)` keeps comparison orientation consistent. For a candidate with opposite sign, the shader compares direct pixel-to-neighbor distance. For matching sign, it compares the distance from the current pixel to the candidate's stored seed coordinate. This preserves both proximity and inside/outside sign.

Mask gate. The JFA shader samples the expanded mask. Pixels outside the mask are cleared to zero, while pixels inside participate. Four expansion passes provide a border so outlines and glows can extend outside the exact silhouette.

```
Conceptual field record: vec3(nearestSeedX, nearestSeedY, signedDistance)
```

Important consequence. The field is screen-space and regenerated every frame. It is not a 3D signed-distance volume and does not measure distance through the mesh in object space. Effects describe the rendered silhouette from the current camera view.

Cost behavior. JFA iteration count grows logarithmically with the initial step. Increasing `thickness` changes the visual radius and reduces mask resolution, trading detail for performance.

6. GLSL shader responsibilities

Shader	Inputs	Core operation	Output
model.vert	position, normal, viewProj, viewportSize, imagePan, imageZoom	Transforms model vertices and applies screen pan/zoom; forwards normals.	gl_Position and vNormal.
lit.frag	vNormal, lightDir	Hemisphere term from normal Y plus diffuse directional term; flips back-facing normals.	Opaque gray-blue color.
mask.frag	vNormal	Ignores lighting and writes a solid value for covered fragments.	White silhouette in R8.
seed.frag	negative	Writes fragment coordinates and signed large seed distance.	Initial field seeds.
quad.vert	gl_VertexID	Builds a full-screen triangle without a vertex buffer.	Clip position and UV.
seed_quad.frag	gl_FragCoord	Writes far sentinel seed everywhere.	Initialized field.
expand.frag	source	Checks eight neighbors and sets output if any is nonzero.	Dilated mask.
jfa.frag	source, mask, step	Checks eight jump neighbors and retains the nearest valid signed seed.	Updated field record.
effect.frag	map, mask, mode, inside, thickness, color, time	Maps signed distance to one of eight effects with smooth alpha edges.	Alpha-blended color.

Effect modes in `effect.frag`

UI name	Mode	Behavior
Mask	-1	Displays the sampled expanded mask as grayscale.
Coordinate	0	Visualizes stored seed coordinates normalized by field size.
SDF	1	Displays absolute distance normalized by thickness.
Outline	2	Colored band around the signed boundary.
Glow	3	Distance-based alpha for a soft colored halo.
Pulse	4	Time-varying sinusoidal motion with derivative-based smoothing.
Halftone	5	3x3 neighborhood of cell centers forms a dot-like pattern.
Rings	6	Periodic distance bands produce concentric rings.

Derivative-aware edges. The helper `fw` estimates screen-space derivative magnitude using `dFdx` and `dFdy`. Pulse and other soft boundaries use this estimate to reduce aliasing.

7. Composition, UI, and interaction

Effect compositing. `renderEffectPass` binds the completed field and mask, supplies time/thickness/color/inside uniforms, and draws a full-screen triangle with alpha blending. The scene remains visible wherever the effect shader discards or produces low alpha.

Inside/outside. C++ supplies inside = -1 when inside mode is enabled and +1 otherwise. The effect shader multiplies signed distance by this value, reversing which side receives the effect.

Dear ImGui panel. `renderUi` displays model name/status, effect combo, Inside checkbox, Thickness slider from 1 to 250, color editor, auto-rotation checkbox, FSAA toggle when supported, frame timing, and draw-call statistics. The file dialog accepts OBJ, glTF, GLB. The ImGui Win32 handler is consulted before custom input.

Input	Action
Left drag	Orbit: changes yaw and clamped pitch.
Right drag	Pan: changes imagePanX/imagePanY in screen pixels.
Mouse wheel	Zooms imageZoom between 0.25 and 4.0.
M / Shift+M	Next / previous effect.
I	Toggle inside/outside effect orientation.
Up / Down	Adjust thickness by 0.25; Shift uses increments of 2.
C	Cycle through five effect colors.
Space	Toggle auto-rotation.
Esc / close window	Exit the application.

Frame timing. The loop uses `QueryPerformanceCounter`, maintains a moving average over the first 60 samples, reports milliseconds/FPS in the UI, and updates the window title approximately every 250 ms. Auto-rotation advances yaw based on elapsed time and stops while the user is orbit-dragging.

8. Resolve, presentation, and cleanup

When MSAA is available and not disabled, scene and effect rendering use `multisampleOutput`. `resolvePass` blits its color attachment into `presentationTarget`. The UI is then drawn into the presentation target after the resolve, so it is not itself multisampled by this path.

`presentPass` blits the presentation target into the default framebuffer. `SwapBuffers` presents the Win32 back buffer. Cleanup shuts down ImGui backends, destroys the ImGui context, releases the WGL context and device context, and exits.

Resize behavior. WM_SIZE calls `windowResize`, which updates width/height and resizes presentation, field, and multisample targets. Mask targets resize lazily inside `renderMaskPass` because their dimensions depend on thickness.

Framebuffer format rationale

Data	Format	Reason
Scene/presentation	RGBA8	Sufficient for display color and blend composition.
Depth	DEPTH_COMPONENT24	Depth testing for scene and visible seed selection.
Mask	R8	Binary coverage needs one channel and low precision.
Distance field	RGBA32F	Seed coordinates and signed distances need floating-point precision.

OpenGL state discipline. Each major pass explicitly toggles depth testing, blending, and framebuffer binding for its needs. Full-screen passes use a triangle rather than a quad, avoiding diagonal seam/coverage issues and requiring no vertex data.

9. Limitations and practical implications

The README documents several intentional limits that are important when interpreting behavior:

Limitation	Impact
Geometry-only import	Materials, texture maps, and material colors are ignored. Imported meshes receive the renderer’s fixed lighting color.
No HDR environment	The port uses simple directional plus hemisphere lighting rather than image-based lighting from the original demo.
Triangles only	Non-triangle glTF primitives are rejected. OBJ faces are triangulated.
No animation	glTF animation channels are not evaluated; imported scenes are static.
No sparse accessors	Sparse glTF accessors produce a loader error.
Screen-space field	Effects are view-dependent silhouette effects, not object-space distances.
Reduced mask resolution	Large thickness values lower mask detail and can make boundaries coarse.
Windows-only	The host depends on Win32/WGL and desktop OpenGL.

Edge cases handled. The code validates empty/invalid geometry, non-finite positions, invalid glTF accessors and buffer ranges, index range errors, invalid node references, node cycles, invalid default scenes, unsupported extensions, shader failures, and incomplete framebuffer creation. A failed model load does not discard the current mesh.

Interpretation of thickness. Thickness is both a visual distance scale and a pipeline parameter. It controls reduced mask dimensions, the initial JFA step, and width/radius calculations in the effect shader. It therefore affects quality, performance, and appearance simultaneously.

10. Build and run guide

Preferred scripted build. From the project directory, run `build.bat`. It deletes the generated build directory, regenerates an x64 Visual Studio solution with CMake, and builds Debug. Close Visual Studio first if generated files are in use.

Alternative Developer PowerShell commands

```
cmake -S . -B build -A x64
cmake --build build --config Debug
```

The executable expects a sibling `shaders` directory. CMake's post-build copy command creates this arrangement. Runtime shader loading resolves the executable location with `GetModuleFileNameW` and appends `shaders\<file>`.

Runtime checklist

Check	Expected result
Open the executable	A 1280x720 window appears with the torus knot and ImGui panel.
Change Effect	The overlay switches among mask, coordinate, SDF, outline, glow, pulse, halftone, and rings.
Change Thickness	Effect width changes and mask/JFA working resolution updates.
Orbit/pan/zoom	The model rotates in 3D; pan and zoom adjust the projected image.
Load model	Valid OBJ/glTF/GLB replaces the mesh; invalid input leaves the old mesh visible.
Disable FSAA	The app bypasses the multisample target when supported.

11. End-to-end data-flow summary

The shortest accurate mental model of the application is:

1. Create a Win32 window and OpenGL 3.3 core context. Load OpenGL entry points and compile shaders.

2. Build the fallback torus knot or import a validated triangle mesh. Center and scale it, then upload positions and normals.

3. Calculate the orbit camera, projection/view matrix, light direction, and MSAA state.

4. Render the lit mesh into the scene target and render it again into a reduced binary mask.

5. Run four mask dilation passes to create a padded coverage region.

6. Initialize the field with a far sentinel and render signed surface seeds using depth testing.

7. Run ping-pong JFA passes. Each checks eight neighbors at the current jump distance and keeps the nearest signed seed.

8. Run the selected effect shader. It reads signed distance, applies inside/outside orientation, thickness, color, and time, then alpha blends over the scene.

9. Resolve MSAA if enabled, draw ImGui, blit the presentation target to the window, and swap buffers.

> **In one sentence:** this is a view-dependent GPU image-processing pipeline that converts a rendered mesh silhouette into a signed distance field and uses that field to draw stylized effects around the object.

Reference points in the current source

Function	Approx. line	Role
`makeMesh`	588	Procedural fallback geometry.
`normalizeModel`	807	Center and scale imported geometry.
`loadObj` / `loadGltf`	867 / 1077	Asset parsing and validation.
`renderScenePass`	1353	Lit scene render.
`renderMaskPass`	1377	Reduced silhouette.
`renderSeedPass`	1417	Signed seed initialization.
`jumpFloodPass`	1442	Nearest-seed propagation.
`renderEffectPass`	1474	Effect visualization/composite.
`render`	1523	Per-frame pass orchestration.
`renderUi` / `wndProc`	1549 / 1658	Controls, status, input.
`wWinMain`	1852	Startup, frame loop, shutdown.

Generated from the source tree on 2026-10-20. The explanation reflects the code and README as inspected on the main branch.